

SECURE CODING REVIEW REPORT

CodeAlpha Cyber Security Internship

Task 3: Secure Coding Review

Submitted By: Abdullah Saeed

Student ID: CA/DF1/197436

Domain: Cyber Security

Organization: CodeAlpha

1. Introduction

Secure coding is the practice of writing software that protects applications from security vulnerabilities and cyber threats. Poor coding practices can expose applications to attacks such as unauthorized access, password theft, data leakage, and code manipulation.

This project reviews a simple Python-based login application to identify common security weaknesses and demonstrates how these issues can be resolved by implementing secure coding practices.

2. Objectives

The objectives of this project are:

- Review an application's source code for security vulnerabilities.
 - Identify insecure coding practices.
 - Compare vulnerable and secure implementations.
 - Apply secure coding techniques to improve application security.
 - Recommend best practices for writing secure software.
-

3. Application Overview

The application selected for this review is a **Python-based Login System**.

Two versions of the application were developed:

- **Vulnerable Login System**
- **Secure Login System**

The vulnerable version intentionally contains common security weaknesses, while the secure version demonstrates recommended security improvements using Python and the **bcrypt** library.

4. Vulnerable Code Analysis

The vulnerable login system contains several insecure coding practices that could be exploited by attackers.

The following weaknesses were identified:

- Hardcoded username and password.
- Plain text password storage.
- Plain text password comparison.
- No password hashing.
- No protection against repeated login attempts.
- Limited input validation.

These weaknesses reduce the overall security of the application and make it easier for attackers to compromise user credentials.

5. Security Vulnerabilities Identified

Hardcoded Credentials

The username and password are directly written inside the source code.

Risk:

If an attacker gains access to the source code, the credentials are immediately exposed.

Plain Text Password Storage

Passwords are stored in plain text instead of using secure hashing techniques.

Risk:

Anyone who accesses the application code can read user passwords.

Plain Text Password Verification

The vulnerable application compares passwords directly.

Risk:

This approach provides no protection if credentials are leaked.

Weak Input Validation

The application performs minimal validation of user input.

Risk:

Poor validation may increase the risk of unexpected behavior or future vulnerabilities.

Information Disclosure

The application provides login-related information without implementing additional security controls.

Risk:

Attackers may gain useful information during authentication attempts.

6. Secure Code Improvements

The secure version implements several security enhancements.

Password Hashing

Passwords are protected using the **bcrypt** hashing algorithm instead of plain text.

Secure Password Verification

The application verifies passwords using **bcrypt.checkpw()**, ensuring secure password comparison.

Improved Authentication

Authentication is performed without exposing sensitive password information.

Generic Error Messages

The secure application displays a generic login failure message instead of revealing unnecessary details.

Enhanced Security

Overall application security is significantly improved through secure password handling and better authentication practices.

7. Comparison Between Vulnerable and Secure Code

Feature	Vulnerable Code	Secure Code
Password Storage	Plain Text	Hashed using bcrypt
Password Verification	Direct Comparison	Secure Hash Verification
Credential Security	Hardcoded	Protected with Hashing
Authentication	Basic	Improved
Security Level	Low	High

8. Best Practices for Secure Coding

The following practices are recommended for developing secure software:

- Never store passwords in plain text.
 - Use strong password hashing algorithms such as bcrypt.
 - Validate all user input.
 - Use generic error messages during authentication.
 - Avoid hardcoded credentials.
 - Keep software libraries updated.
 - Follow the Principle of Least Privilege.
 - Perform regular code reviews and security testing.
-

9. Results

The secure version successfully addresses the major security weaknesses found in the vulnerable application.

By implementing password hashing with **bcrypt**, secure password verification, and improved authentication practices, the application's security is significantly enhanced.

The comparison clearly demonstrates the importance of following secure coding standards during software development.

10. Conclusion

Secure coding is an essential part of software development and cybersecurity. Applications that contain insecure coding practices are more vulnerable to cyberattacks and unauthorized access.

This project successfully reviewed a vulnerable login system, identified common security weaknesses, implemented appropriate security improvements, and demonstrated the benefits of secure coding practices. The project fulfills the objectives of the CodeAlpha Cyber Security Internship Task 3.

11. References

- Python Documentation – <https://www.python.org/>
- bcrypt Documentation – <https://pypi.org/project/bcrypt/>
- OWASP Secure Coding Practices – <https://owasp.org/>
- OWASP Top 10 – <https://owasp.org/www-project-top-ten/>